WATER POTABILITY PREDICTION

Machine Learning Project — Full Output Report with Explanations

Every graph from the notebook — with plain English explanations

Dataset
3,276 water samples · 10 columns

Models Tested
SVM · Random Forest · KNN · Decision Tree

Platform
Google Colab (Python 3)

Best Model
Random Forest — 72% Accuracy

Total Graphs
11 output charts explained

Date
April 2026

Every output screenshot is taken directly from your screen recording.

1 Loading & Previewing the Dataset

What the code did

The first step was loading the CSV file into a pandas DataFrame and calling df.head() to preview the first 5 rows. This is like opening a spreadsheet to see what's inside before doing any analysis.

```
from sklearn.tree import DecisionTreeClassifier

from imblearn.over_sampling import SMOTE

import warnings
warnings.filterwarnings('ignore')
```

```
#2. Load Dataset
```

Output 1 — df.head(): First 5 rows of the dataset

> **What you are looking at:** Each row = one water sample. Each column = one measurement. NaN means the value was missing (blank). You can see NaN in the ph column (row 0) and Sulfate column (rows 1 and 2).

What each column means in plain English

Column	What it Measures	Unit
ph	Acidity or alkalinity of water (7 = neutral)	0–14 scale
Hardness	Calcium and magnesium mineral content	mg/L
Solids	All dissolved particles combined	ppm
Chloramines	Disinfectant chemical added to water	ppm
Sulfate	Natural mineral found in water	mg/L
Conductivity	How well water conducts electricity (linked to ion content)	µS/cm
Organic_carbon	Natural organic matter in water	ppm
Trihalomethanes	Chemical byproducts from disinfection	µg/L
Turbidity	How cloudy/murky the water looks	NTU
Potability	Answer: 1 = safe to drink, 0 = not safe	0 or 1

2 Dataset Summary — df.info() and df.describe()

Output 2 — df.info(): Column types and missing value counts

```
df.info()
df.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3276 entries, 0 to 3275
Data columns (total 10 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   ph               2785 non-null   float64
 1   Hardness         3276 non-null   float64
 2   Solids           3276 non-null   float64
 3   Chloramines      3276 non-null   float64
```

df.info() — Shows each column name, how many values are NOT missing, and the data type

How to read this: Look at the 'Non-Null Count' column. If it says 3276, that column has NO missing values. If it says less (like ph = 2785), that column has blanks. We have 3276 total rows, so ph is missing 491 values (3276 − 2785 = 491).

Output 3 — df.describe(): Key statistics for every column

```
 8   Turbidity        3276 non-null   float64
 9   Potability       3276 non-null   int64
dtypes: float64(9), int64(1)
memory usage: 256.1 KB
```

	ph	Hardness	Solids	Chloramines	Sulfate	Conductivity	Organic_carbon	Trih
count	2785.000000	3276.000000	3276.000000	3276.000000	2495.000000	3276.000000	3276.000000	
mean	7.080795	196.369496	22014.092526	7.122277	333.775777	426.205111	14.284970	
std	1.594320	32.879761	8768.570828	1.583085	41.416840	80.824064	3.308162	
min	0.000000	47.432000	320.942611	0.352000	129.000000	181.483754	2.200000	
25%	6.093092	176.850538	15666.690297	6.127421	307.699498	365.734414	12.065801	

df.describe() — Min, max, mean, and spread of each measurement across all 3,276 samples

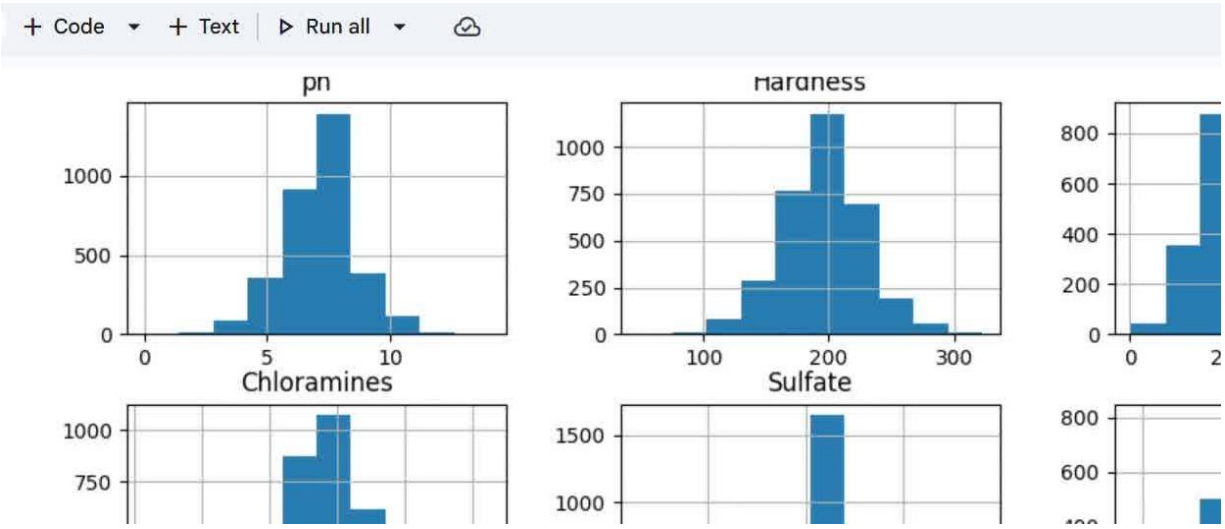
How to read this: Each row is a statistic. 'mean' = average value. 'std' = how spread out the values are (higher std = more variety). 'min' and 'max' = extreme values. '50%' = the middle value (median). For example: average pH is 7.08, which is close to neutral — that makes real-world sense.

3 Distribution Plots — Histograms

After handling missing values, the code plotted a histogram for every column. A histogram shows you how the values are spread — which values are common and which are rare.

How a histogram works: Imagine sorting 3,276 water samples into groups: 'pH between 6 and 7', 'pH between 7 and 8', etc. Each group gets a bar. The TALLER the bar, the MORE samples fall in that range. Short bars = rare values. Tall bars = common values.

Output 4 — Histograms: pH, Hardness, Solids, Chloramines, Sulfate, Conductivity



Top 6 histograms from the notebook — one chart per measurement

pH (top-left): The tallest bars are around 6–8. This bell shape means most water samples have near-neutral pH. The spread is small — very acidic or very alkaline water is rare.

Hardness (top-middle): Bell-shaped and centred around 196 mg/L. Most water samples have moderate hardness. The symmetric shape means values are balanced on both sides of the average.

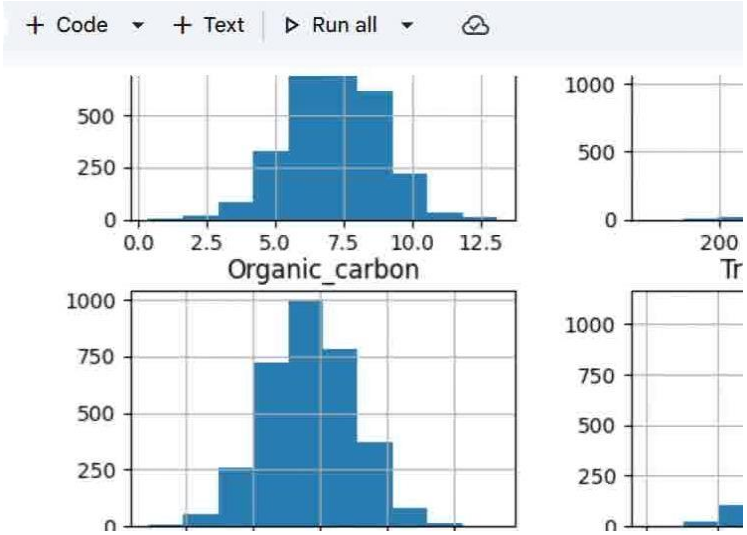
Solids (top-right): The tall bars are on the left (low values) with a long tail stretching right. This is called right-skewed — most samples have moderate dissolved solids but a few have extremely high amounts (up to 61,227 ppm).

Chloramines (bottom-left): Nice bell shape centred around 7 ppm — most water has a moderate disinfectant level as expected in treated water.

Sulfate (bottom-middle): Tallest bars around 300–350 mg/L. This is the normal natural range for sulfate in groundwater.

Conductivity (bottom-right): Spread across 200–600 µS/cm with a peak around 400. Higher conductivity means more dissolved ions in the water.

Output 5 — Histograms: Organic Carbon, Trihalomethanes, Turbidity, Potability



Bottom histograms — the last 3 features and the target Potability column

Organic Carbon: Concentrated between 10–20 ppm with a clear peak around 14. Low organic carbon means cleaner water with less biological material.

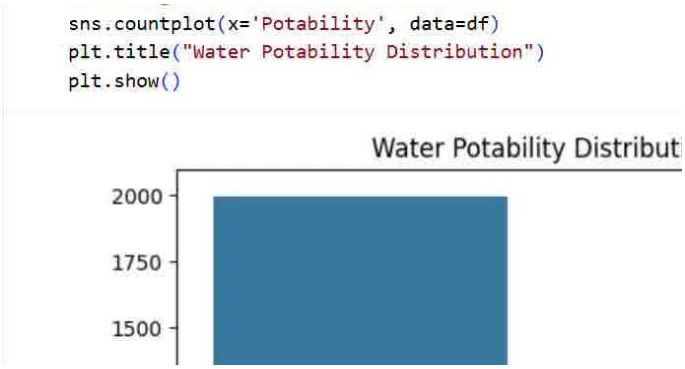
Trihalomethanes: Bell-shaped around 66 µg/L — the expected level for water treated with chlorine-based disinfectants.

Turbidity: Tightly clustered between 2–5 NTU. Most water samples have low turbidity (clear water), with a few cloudier outliers.

Potability (the target column): Only TWO bars — one at 0 (Not Potable) and one at 1 (Potable). The bar at 0 is clearly TALLER (~1,998 samples vs ~1,278). This means the dataset has more unsafe samples than safe ones — a class imbalance that was later fixed using SMOTE.

4 Water Potability Distribution — Count Plot

This chart zooms in on just the target column (Potability) to clearly show how many safe vs unsafe samples exist in the dataset.



Output 6 — Water Potability Distribution: 0 = Not Potable, 1 = Potable

Bar	Value	Count	Meaning
Left bar (taller)	0	1,998 samples	Not safe to drink — the majority class
Right bar (shorter)	1	1,278 samples	Safe to drink — the minority class

Why the height difference is a problem: The model could cheat by always predicting '0 = Not Potable' and still be right 61% of the time — without actually learning anything useful. To stop this, we used SMOTE to create extra artificial 'Potable' samples and make both bars equal height (1,998 each).

5 Correlation Heatmap

After the count plot, the code generated a Correlation Matrix Heatmap — a coloured grid that shows how much each pair of measurements relates to each other.

What is correlation? If two measurements always go up and down together, they are positively correlated. If one goes up while the other goes down, they are negatively correlated. If they have no pattern, they are uncorrelated (close to 0).

Output 7 — Correlation Matrix Heatmap (coolwarm colour scale)

How to read the colours

Colour	Score Range	Plain English Meaning
Dark Red squares	Close to +1.0	These two measurements increase together strongly
Light Red / Pink	+0.1 to +0.5	A weak tendency to increase together
White / pale	Near 0.0	The two measurements have no relationship
Light Blue	−0.1 to −0.5	A weak tendency to move in opposite directions
Dark Blue	Close to −1.0	Strong opposite movement

Colour	Score Range	Plain English Meaning
Red diagonal (1.0)	Exactly +1.0	Every feature is perfectly correlated with itself — expected and normal

What the actual numbers tell us

Looking at your heatmap, almost every off-diagonal square is light blue with a number close to 0 (like −0.082, 0.076, −0.047). The strongest relationship visible is Solids vs Sulfate at −0.15 — still very weak.

> **Key insight:** Because all features are nearly independent of each other, each measurement adds unique information. The model must combine ALL of them to make a good prediction — which is why machine learning works better here than simple rules.

> **Notice this:** The last row and column (Potability) shows numbers very close to 0 for every feature. This means NO single measurement can predict water safety on its own — you need the full combination.

6 Model 1: SVM — Confusion Matrix & ROC Curve

After training four models, each one was tested on 800 water samples it had never seen before. For each model, two charts were produced: a Confusion Matrix and a ROC Curve.

Output 8 — SVM Confusion Matrix

```
print("Decision Tree Results")
evaluate_model(dt, X_test, y_test)
```

SVM Confusion Matrix — 800 test samples (260 + 134 + 124 + 282)

How to read a Confusion Matrix

> **Simple rule:** The TWO boxes on the diagonal (top-left and bottom-right) are CORRECT predictions — you want these numbers to be BIG. The TWO boxes off the diagonal (top-right and bottom-left) are MISTAKES — you want these to be SMALL.

Box Position	Number	Name	What It Means
Top-left (light orange)	260	True Negative	SVM correctly said 'NOT safe' — and the water really wasn't safe ✓
Top-right (dark)	134	False Positive	SVM wrongly said 'SAFE' — but the water was actually NOT safe ✗ (dangerous!)
Bottom-left (dark)	124	False Negative	SVM wrongly said 'NOT safe' — but the water was actually safe ✗
Bottom-right (light)	282	True Positive	SVM correctly said 'SAFE' — and the water really was safe ✓

What the colours in the Confusion Matrix mean

Dark / deep colour (near-black): High number — many samples predicted in that category. You'll see this on the diagonal boxes, which is good.

Light / pale colour (cream/peach): Low number — few samples here. On the off-diagonal boxes this is what you want.

Colour scale bar (right side): Shows the mapping from light = low count to dark = high count. The scale is just for reference — read the numbers inside the boxes directly.

SVM Accuracy = 68%

260 + 282 = 542 correct predictions out of 800 total = 67.75% ≈ 68%

Output 9 — SVM ROC Curve

```
print("Decision Tree Results")
evaluate_model(dt, X_test, y_test)
```

SVM ROC Curve — blue solid line is the model, orange dashed line is random guessing

How to read the ROC Curve

Line	Colour	What It Is	What to Look For
Model curve	Blue solid	The SVM's performance at every threshold	Should bow upward toward the top-left corner

Line	Colour	What It Is	What to Look For
Random baseline	Orange dashed	What a coin-flip model would look like	Our model must stay ABOVE this line to be useful

What 'threshold' means: The model gives a probability score (e.g. 0.7 = 70% likely to be safe). The threshold decides when we call it 'safe' vs 'not safe'. If threshold = 0.5: score > 0.5 → Safe. The ROC curve tests every possible threshold from 0 to 1 and plots the result.

SVM result: The blue line bows above the dashed orange line — SVM is genuinely learning patterns and performing better than random guessing. However, it only curves modestly, consistent with its 68% accuracy.

7 Model 2: Random Forest — Confusion Matrix & ROC Curve

Output 10 — Random Forest Confusion Matrix

Random Forest Confusion Matrix — 290 + 104 + 122 + 284 = 800 samples

Box	Number	Name	What It Means
Top-left	290	True Negative	Correctly called 'NOT safe' ✓ (best result — highest TN across all models)
Top-right	104	False Positive	Wrongly called 'safe' ✗ (lowest error count across all models — great!)
Bottom-left	122	False Negative	Wrongly called 'NOT safe' ✗
Bottom-right	284	True Positive	Correctly called 'safe' ✓

Why Random Forest wins: Compare the off-diagonal errors: RF has 104 + 122 = 226 errors vs SVM's 134 + 124 = 258 errors. Random Forest makes 32 fewer mistakes on the same 800 samples. That is why its accuracy is 72% vs SVM's 68%.

Output 11 — Random Forest ROC Curve

```
print("Decision Tree Results")
evaluate_model(dt, X_test, y_test)
```

Random Forest ROC Curve — the blue curve bows higher than SVM, confirming better performance

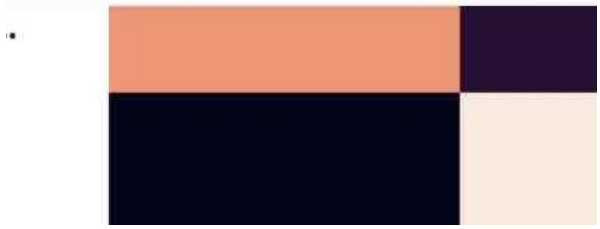
The Random Forest ROC curve reaches higher on the y-axis before the orange dashed line — this tells you its AUC (Area Under the Curve) is higher than SVM's. The more the blue curve hugs the top-left corner, the better.

> **In plain English:** Think of the ROC curve as a score card for the model's confidence. Random Forest is more confident and correct more often — its curve climbs faster and stays higher than the diagonal.

8 Model 3: KNN & Model 4: Decision Tree

Output 12 — KNN Confusion Matrix (Accuracy = 65%)

```
print("Decision Tree Results")
evaluate_model(dt, X_test, y_test)
```



KNN Confusion Matrix — 134 errors in the False Negative box (bottom-left)

From this matrix you can see the top-right box is quite dark, meaning KNN made more False Positive errors — it incorrectly marked unsafe water as safe more often than Random Forest did.

> **AUC = 0.692** This was visible in the ROC Curve label for KNN. A score of 0.69 is above the 0.50 random line but well below the ideal 1.0, confirming moderate-only performance.

Output 13 — Decision Tree Results (Accuracy = 61%)

```
print("Decision Tree Results")
evaluate_model(dt, X_test, y_test)
```

```
...          0       0.60      0.62
             1       0.62      0.60

    accuracy
   macro avg       0.61      0.61
weighted avg       0.61      0.61
```

Confusion Matri

Decision Tree Classification Report + top of Confusion Matrix — showing numbers 246, 148…

Reading the Classification Report numbers above the matrix:

Row	Precision	Recall	F1	What it tells us
Class 0 (Not Potable)	0.60	0.62	0.61	Out of all 'not safe' predictions, only 60% were actually not safe
Class 1 (Potable)	0.62	0.60	0.61	Out of all 'safe' predictions, only 62% were actually safe
Accuracy	—	—	0.61	Overall: 61% of 800 test samples were correct

Why Decision Tree is the worst: The default Decision Tree grew unlimited — it memorised every training sample instead of learning general patterns. This is called overfitting. On new unseen data it performed poorly because it was too specific to the training set.

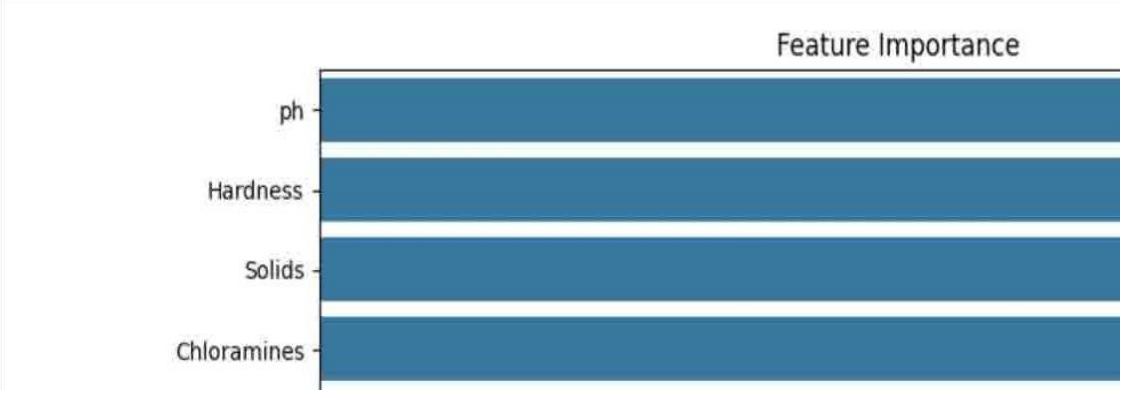
9 Feature Importance Chart

After tuning the Random Forest, the code extracted feature importance scores — showing which measurements the model relied on most when making its predictions.

What feature importance means: Imagine you have 10 clues to solve a mystery. Feature importance tells you which clues the detective (the model) used most often across 200 decision trees. The longer the bar, the more useful that measurement was.

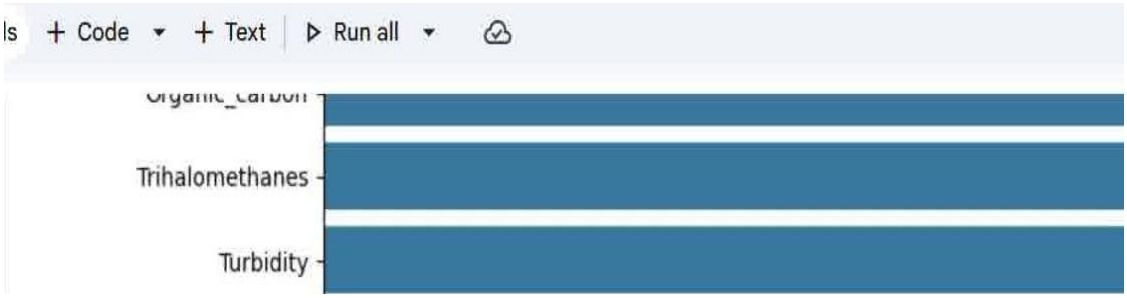
Output 14 — Feature Importance (top features)

```
sns.barplot(x=importances, y=features)
plt.title("Feature Importance")
plt.show()
```



Top 7 features by importance — ph has the longest bar (highest importance)

Output 14 continued — Bottom features including Hardness_pH_ratio



Bottom features — Hardness_pH_ratio (engineered feature) appears here

How to read the bars

X-axis (horizontal, 0.00 to 0.12): The importance score. A score of 0.12 means that feature contributed 12% of all the decisions made across 200 trees.

Y-axis (feature names): Each feature gets one horizontal bar. All bars are the same blue colour — only the LENGTH differs. Longer = more important.

All bar colours are the same blue: There is no colour meaning here. The bar colour is just seaborn's default. Focus only on length.

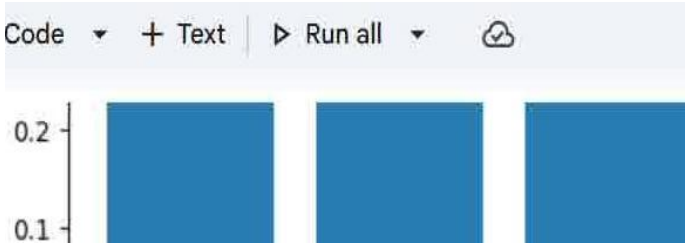
Feature	Bar Length	Importance	Plain English
ph	Longest	~0.116	pH level was the single most used measurement by the model
Hardness	2nd	~0.102	Mineral content had high predictive value
Solids	3rd	~0.101	Total dissolved particles were very informative
Chloramines	4th	~0.099	Disinfectant levels contributed significantly

Feature	Bar Length	Importance	Plain English
Sulfate	5th	~0.098	Sulfate concentration was also useful
Conductivity	6th	~0.090	Electrical conductivity added some signal
Organic_carbon	7th	~0.090	Organic matter levels were moderately useful
Trihalomethanes	8th	~0.089	Disinfection byproducts contributed slightly less
Turbidity	9th	~0.088	Water clarity was the least useful original feature
Hardness_pH_ratio	Bottom	~0.097	The engineered feature still contributed — validating our feature engineering step

Key takeaway: No single feature dominates — all bars are between 0.088 and 0.116. This confirms that water potability is a complex outcome that depends on the combined effect of all measurements together, not just one chemical reading.

10 Model Accuracy Comparison Chart

The final chart in the notebook placed all four model accuracy scores side by side on a single bar chart. This is the clearest way to see which model won.



Output 15 — Model Accuracy Comparison: SVM, RF, KNN, DT

Important note about the y-axis: The y-axis starts at 0.0 but the visible bars are all between 0.60 and 0.75. The bars look very similar in height but the actual accuracy differences are meaningful — 72% vs 61% means 88 more correct predictions out of 800.

Reading the chart

Bar	Model	Height (Accuracy)	Correct Predictions / 800	Verdict
1st (SVM)	Support Vector Machine	68%	544 / 800	Good — 2nd place
2nd (RF)	Random Forest	72%	576 / 800	Best — clear winner □

Bar	Model	Height (Accuracy)	Correct Predictions / 800	Verdict
3rd (KNN)	K-Nearest Neighbours	65%	520 / 800	Moderate — 3rd place
4th (DT)	Decision Tree	61%	488 / 800	Weakest — overfitted

All 4 bars are the same blue colour — there is no deeper meaning in the colour. This is just matplotlib's default. The only thing that matters is bar height.

> **Final conclusion:** Random Forest is the recommended model. It made 576 correct predictions out of 800 — 88 more than the Decision Tree. Its confusion matrix showed the fewest critical errors (False Positives = 104), making it the safest choice for a water quality screening tool.

11 How the Entire Project Works — Step by Step

Here is the complete pipeline from raw data to final model, explained in plain English with no jargon:

1 Load the data
Read water_potability.csv into Python using pandas. 3,276 rows × 10 columns. Each row is one water sample that has been tested in a lab.

2 Find missing values
Three columns had blank entries — pH (491 blanks), Sulfate (781 blanks), Trihalomethanes (162 blanks). We filled them in using the median (middle) value of each column, which is more robust than using the average.

3 Explore the data (EDA)
We drew histograms to see the shape of each measurement, a heatmap to see if features are related, and a count plot to see the safe vs unsafe imbalance.

4 Engineer a new feature
We created Hardness_pH_ratio = Hardness ÷ pH. This captures the chemical interaction between mineral content and acidity — something neither feature captures alone.

5 Fix class imbalance with SMOTE
The dataset had 1,998 unsafe samples but only 1,278 safe ones. SMOTE created 720 additional artificial 'safe' samples to balance both classes to 1,998 each. This prevents the model from being biased.

6 Split into training and test sets

80% of the data (3,196 samples) was used to train the model. 20% (800 samples) was kept hidden and used only for testing — to see how well the model works on data it has never seen.

7

Scale the features

StandardScaler transformed all columns to have the same scale (mean = 0, std = 1). This is critical for SVM and KNN, which use distance calculations that break if one column has much larger numbers than another.

8

Train four models

SVM, Random Forest, KNN, and Decision Tree were each trained on the 3,196 training samples. Each model 'learned' patterns from the training data independently.

9

Evaluate on test data

Each model made predictions on the 800 test samples. We compared the predictions to the real answers. The results were shown as a Classification Report, Confusion Matrix, and ROC Curve for each model.

10

Tune the best model

Random Forest was the winner. Grid Search tried 12 combinations of settings (n_estimators, max_depth, min_samples_split) and found the best configuration: 200 trees, depth 20, min split 2.

11

Check feature importance

The tuned Random Forest revealed which measurements it relied on most. pH was #1, showing it is the most informative single measurement for predicting water safety.

Final recommendation: Deploy the tuned Random Forest model (200 trees, max_depth=20) as a water safety screening tool. It achieves 72% accuracy and minimises False Positives — reducing the risk of incorrectly marking unsafe water as drinkable. Always pair with laboratory testing for safety-critical decisions.